SAGAR NETRA

Underwater Marine Debris Detection and Reporting System

Project Review

Submitted by

Kabin Mukesh R 24BCT0117

Adithya P 24BCT0135

Kavin Kumaran 24BCT0144

Introduction

Sagar Netra means “eye of the ocean”. It is a system that helps people report rubbish they find in the sea, especially abandoned fishing nets, and helps the coastal authorities decide what to clean up first. A diver, a fisherman or a survey team takes a photo, uploads it with the location, and our trained AI model looks at the picture and marks every piece of debris it can see. The report, the photo and the model’s findings are saved in a database, shown on a map, and made available to the authority that can send a team to remove it.

The project brings together three things: a deep-learning model that can recognise objects underwater, a web application where people sign in and submit reports, and a cloud database that keeps everything organised so it can be monitored and acted on.

The Problem

Fishing nets that are lost or thrown away at sea are called ghost nets. They keep catching fish, turtles and dolphins for years, long after anyone is using them. Along with plastic bags, bottles and other waste, they damage coral reefs and can tangle boat propellers. India has a very long coastline and lakhs of fishermen who see this debris almost every day, but there is no simple way for them to tell anyone who can do something about it.

Even when a sighting is reported, it is hard to know how serious it is. A net is thin and spread out, so it is easy to miss in murky water, and a single plastic bottle and a huge drifting net can look the same in a list of complaints. Survey teams with underwater cameras also collect thousands of photos, far too many to check by hand. So the real gap is not cleaning the sea, it is finding the debris, reporting it quickly and knowing which reports matter most.

What We Set Out to Do

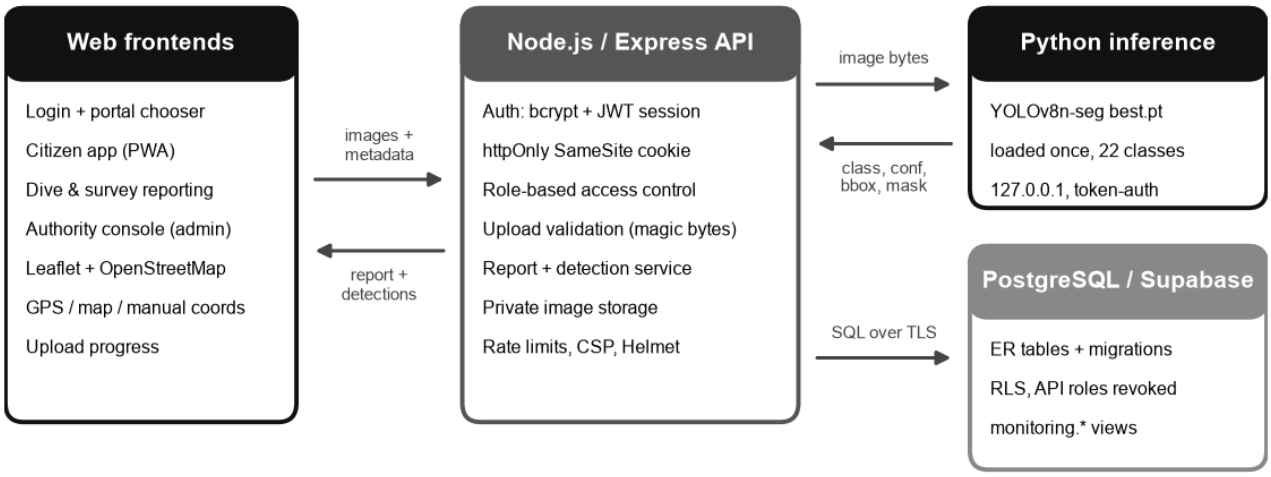
Our goal was to build a working system that:

- recognises debris in underwater photos automatically, including ghost nets and ropes, and outlines each object exactly instead of just drawing a box around it;
- lets citizens, divers and researchers sign in and send reports with photos and a location from GPS or a map;
- stores users, reports and every detection in a proper database designed from our ER diagram;
- gives the coastal authority a map and a list of all reports so they can track and clear them;
- is honest about how accurate the model really is, by testing it on photos it has never seen.

The system has three separate entry points. Citizens use a simple phone-friendly page, divers and researchers use a more detailed page where they can add several photos and dive information, and administrators use an operations console that shows every report on a map.

How the System Works

The system has four parts that work together. The web pages are what users see. Behind them is a Node.js server that checks who the user is, accepts the photos and talks to everything else. A separate Python program runs the AI model, and a PostgreSQL database hosted on Supabase stores all the data. The web pages never touch the model or the database directly; everything goes through the server, which keeps the data safe.



Overall system architecture

A typical report goes like this. The user opens the website, picks the portal that suits them and signs in or creates an account. They choose a location, either from their phone's GPS, by clicking on the map or by typing the latitude and longitude. Then they upload one or more photos and add details such as depth, water temperature, visibility and notes.

The server first checks that each file really is an image and that the location is valid. It saves the photos and creates the report in the database. Each photo is then sent to the model, which returns what it found: the type of object, how confident it is, where it is in the picture and its exact outline. These results are saved against the report and shown to the user straight away, with the objects marked on the photo. The report also appears on the map and in the administrator's console.

If the model is not available for some reason, the report is still saved but clearly marked as "not analysed", and it can be analysed again later. The system never shows made-up results.

To help the authority prioritise, every report gets a severity level. Ghost nets, wreckage and large sheets of plastic are marked high. Small items like a single bottle or can are marked low, and everything else is medium. Fish, plants and other sea life are recorded too, but they are labelled as "not debris" so they do not raise an alarm.

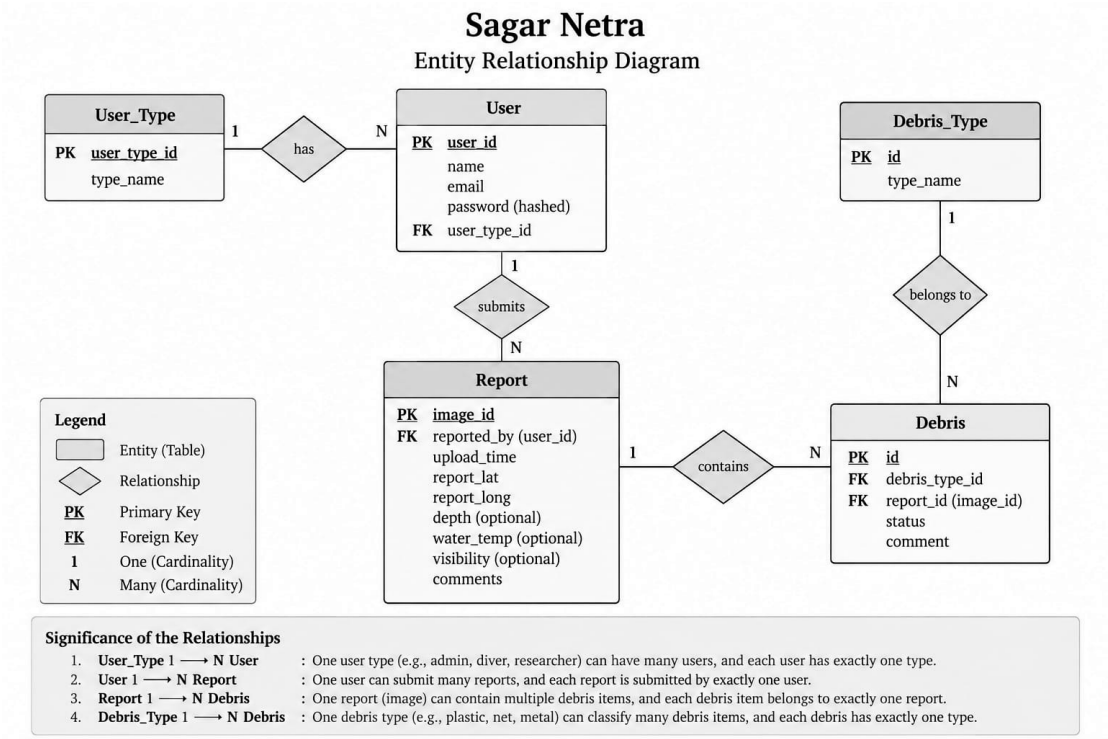
The Data We Used

We trained the model on TrashCan 1.0, a public research dataset from the University of Minnesota. It contains 7,212 real underwater photos taken by remotely operated vehicles, with 12,128 objects outlined by hand. We used 6,065 photos for training and kept 1,147 aside for testing. The dataset has 22 categories: the underwater robot itself, plants, six kinds of sea animals and fourteen kinds of debris such as bottles, bags, cans, pipes, ropes and nets.

We chose the version of the dataset that keeps nets and ropes as separate categories, because those are exactly the things we care most about. The original labels came in a format the model could not read directly, so we wrote a converter that turned all 12,128 outlines into the right format without losing any of them.

Database Design

The database follows the ER diagram below. A user type (admin, diver, researcher or citizen) can have many users. A user can submit many reports. A report can contain many debris detections, and each detection belongs to one debris type.



ER diagram of Sagar Netra

We added two small tables to support the features the app needed: one to hold several photos for the same report, and one to keep track of login sessions so that signing out really logs the user out. The debris type table is filled with the 22 categories the model knows, in the same order the model uses, so results always match the right category. The database also rejects impossible values, such as a latitude above 90 degrees.

Implementation

The AI model

We used YOLOv8n-seg, a small and fast instance segmentation model. Instead of training it from nothing, we started from a version that had already learned general shapes and objects from millions of everyday photos, and then taught it our 22 underwater categories. We first trained it for 15 rounds on a laptop graphics card, and then continued training at a smaller image size until the accuracy stopped improving. The best version was saved and is the one the app uses today.

We picked segmentation rather than plain detection because of the nets. A box drawn around a drifting net is mostly water, but an outline shows how much net is actually there, which is far more useful for planning a cleanup.

The model service

The model runs as a small Python service on the same computer as the server. It loads the model once when it starts, so each photo is analysed quickly, and it only accepts requests from our own server using a secret key. For every photo it sends back the objects it found, their confidence, their position and their outlines, along with a marked-up copy of the image.

The server

The server is written in Node.js with Express. Passwords are never stored as plain text; they are scrambled with bcrypt before saving. When someone signs in they get a secure session, and signing out ends it for good. Each user has a role, and the roles decide what they can do: ordinary users see only their own reports, while administrators can see and manage everything. Nobody can make themselves an administrator when they sign up. The server also limits repeated login attempts, checks every uploaded file and stores photos privately so only the right people can open them.

The database

The data lives in PostgreSQL on Supabase, a cloud database service. The tables are created by script, so the same database can be rebuilt anywhere. We locked the tables so that nothing outside our server can read them, and we added a few read-only summary views so the project admin can watch the number of users, reports and detections from the Supabase dashboard.

The website

The website has a starting page where users choose their portal and sign in, a citizen reporting page that works well on phones, a detailed reporting page for divers and researchers, and the administrator console. Maps use Leaflet with OpenStreetMap, so no paid map key is needed. Users can set their location with GPS, a map click or by typing it, and they see the model’s results, their past reports and their map markers, all loaded from the database.

Results

To see how good the model really is, we tested it ourselves on the 1,147 photos it never saw during training, which together contain about 2,586 objects. The usual way to score this kind of model is mAP, which combines how many objects it finds with how accurate its guesses are. A score of 1.0 would be perfect.

Measure	Finding objects (boxes)	Outlining objects (masks)
mAP at 50% overlap	0.576	0.545
Precision	0.655	0.607
Recall	0.522	0.516

These scores are a little better than our first version of the model, which scored 0.560 and 0.541. The model is very good at large, clear objects: it scores about 0.96 on wreckage, 0.95 on pipes and 0.83 on cans. Ghost nets are harder because they are thin and spread out, and there are only a few examples of them in the data, but the model still finds about 73% of the nets in the test photos.

We also tested the finished system the way a user would, by sending 29 test photos of ghost nets through the running app. It found the net in 25 of them, about 86%, with an average confidence of 93%, and it never reported a net where there wasn't one. Each photo took about 0.13 seconds on an ordinary computer without a graphics card.

Technologies Used

The model was built with Ultralytics YOLOv8 and PyTorch in Python, with OpenCV and NumPy for image handling. The server uses Node.js and Express, with bcrypt for passwords and JSON Web Tokens for sessions. The database is PostgreSQL hosted on Supabase. The website is plain HTML, CSS and JavaScript with Leaflet and OpenStreetMap for the maps. For testing we used Node's built-in test runner with Supertest, and Python's unittest for the model service.

Testing

We wrote 66 automatic tests for the server and 3 for the model service, and all of them pass. They run against a real database and the real model, and check things like signing up and logging in, that passwords are stored safely, that a user cannot see someone else's reports, that wrong locations and fake image files are rejected, that the saved results exactly match what the model produced, and that nothing fake is shown when the model fails. What is still left is testing with real users in the field, using photos from Indian coastal waters.

Project Status

About 90% of the project is complete. All the main parts are built and working together; only field testing and public deployment remain.

Part of the project	Status
Requirements, ER design and dataset preparation	Done
Model training and evaluation	Done
Model service, server and database	Done
Login, user roles and all web pages	Done
Testing with real users	Pending
Deployment on a public website	Pending

Conclusion

Sagar Netra turns a trained AI model into a system people can actually use. Anyone can sign in, upload a photo of underwater debris with its location, and within a fraction of a second see what the model found. Every report is saved in a cloud database and shown on a map, so the coastal authority can see where the problems are and which ones need attention first.

With an outline accuracy of 0.545 on unseen photos and ghost nets found in about 86% of our test images, the system already works well as a complete project. The next steps are to test it with real fishermen, divers and coastal officers, and to put it online so it can be used along the coast.